

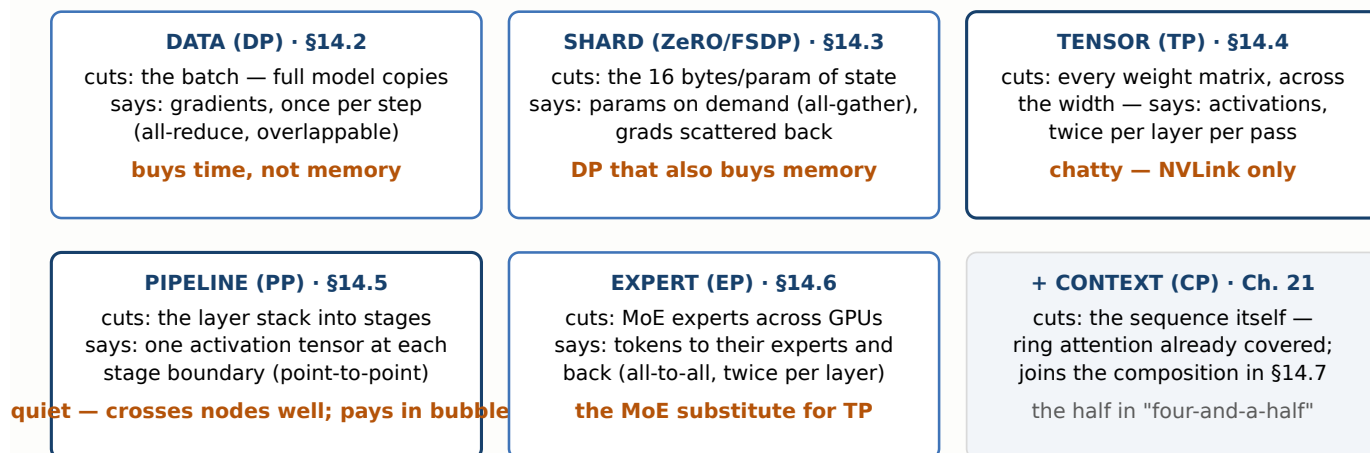
Distributing the model

Data, shard, tensor, pipeline, expert — the four-and-a-half axes of parallelism, and how the frontier composes them

Chapter 4 ended with a sentence that this chapter now has to pay for: a 70B model needs **1.1 TB of training state** before its first token, and the largest GPU you can buy holds 80 GB. That is not a performance problem — it is an existence problem; there is no slow way to run it on one device. And solving existence only starts the clock: even if the state fit, one H100 grinding through a 15T-token diet would need *decades*. Distribution answers both — memory, by cutting the model and its state into pieces; time, by putting thousands of chips on the same loop — and it does so with a small set of ideas that compose like instruction-set primitives. Every impressive-sounding recipe in a frontier paper — "4D parallelism," "16-way pipeline with 64-way expert parallel" — is a sentence in this chapter's grammar. By the end you will read those sentences fluently, write one for your own cluster, and know why the two most instructive frontier runs of recent memory chose *opposite* sentences for the same problem.

One organizing principle carries the whole chapter, and it comes straight from Chapter 4's ladder of wires: **each parallelism axis has a communication signature — how much it talks, how often, and in what pattern — and the art is matching signatures to wires**. Chatty, latency-critical axes live on NVLink inside a node; bulky-but-overlappable axes cross the slower fabric between nodes. Hold that principle and every composition rule in §14.7 becomes obvious instead of memorized.

WHAT EACH AXIS CUTS (and what it must say over the wire)



The whole art: match each axis's communication signature to the wire that can carry it (§14.7).

Figure 14.1 — The axes at a glance. Four cuts of the same training job, plus the sequence cut Chapter 21 already made. Bold-bordered boxes are the chatty ones that demand the fastest wires; the others tolerate distance. Everything else in this chapter is these boxes, opened.

14.1 Two reasons to distribute, and the wires you distribute over

Keep the two motivations separate, because they select different tools. **Throughput distribution** (the model fits; you want the calendar back) is solved by data parallelism alone, and most practitioners live their whole lives here.

Existence distribution (the model does not fit) requires cutting the model itself — sharding, tensor, pipeline, expert — and every cut introduces communication that Chapter 4's wire ladder must carry: ~900 GB/s of NVLink between GPUs inside a node; ~50–100 GB/s of InfiniBand between nodes; an order of magnitude between them. Write those two numbers on the whiteboard before any parallelism meeting. They are the physics the whole chapter negotiates with.

14.2 Data parallelism: the workhorse

Every GPU holds a complete replica; each processes its own micro-batch through Chapter 4's loop; and before the optimizer steps, all replicas average their gradients so every copy takes the identical update. The averaging is the famous **all-reduce**, and its standard implementation — the ring — is worth one paragraph of mechanics because it explains a magic fact: arrange the p GPUs in a ring; in a first phase each sends a chunk of its gradient to its neighbor while receiving and adding a different chunk (after $p-1$ hops, each GPU holds one fully-summed chunk); a second phase circulates the finished chunks. Total traffic per GPU: $2\frac{p-1}{p}$ times the gradient size — *essentially independent of p* .¹ Scaling from 8 to 256 GPUs raises each GPU's sync burden from 1.75× to 1.99× the payload: the ring is why data parallelism scales at all.

The payload, however, is the payload: a 70B model's bf16 gradients are 140 GB, so each step wants ~280 GB through each GPU's network interface — 5.6 naked seconds on a 50 GB/s link, longer than the step itself. The fix is the one trick every DP implementation shares: **overlap**. Gradients for the last layers are ready while backward is still computing the first layers, so the all-reduce streams behind the backward pass, bucket by bucket, and a healthy setup

hides nearly all of it. That is what `DistributedDataParallel` does for you, and why the code is almost insultingly short:

```
# data parallelism: the whole change from Chapter 4's loop
# launch: torchrun --nproc_per_node=8 --nnodes=4 train.py
import torch.distributed as dist
dist.init_process_group("nccl")
rank, local = dist.get_rank(), int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local)

model = Transformer(cfg).cuda()
model = torch.nn.parallel.DistributedDataParallel(model) # buckets + overlapped
loader = DataLoader(ds, sampler=DistributedSampler(ds)) # all-reduce, done

for step, batch in enumerate(loader): # Chapter 4's loop, unchanged:
    loss = model(batch).loss # DDP averages grads during
    loss.backward(); opt.step(); opt.zero_grad() # backward, automatically
    if rank == 0 and step % 50 == 0: log(step, loss.item()) # one rank talks
```

Two facts complete the picture. DP's *global batch* is (micro-batch $\times$ accumulation $\times$ DP degree) — grow the DP degree and you have silently grown the batch, with all of Chapter 17's learning-rate consequences; frontier runs raise accumulation or shrink micro-batches to keep the token count chosen, not accidental. And DP buys *zero* memory: every replica still carries all 16 bytes per parameter. For our 1B model that was fine. For anything past ~7B with activations, it is not — which is where the next section's observation turns DP into something better.

14.3 ZeRO and FSDP: the redundancy was the memory

Look at eight DP replicas through Chapter 4's memory anatomy and an absurdity jumps out: all eight store byte-for-byte identical optimizer state — the same 12 bytes per parameter, eight times. ZeRO's founding move was simply to stop: ² shard the redundant state across the data-parallel group and materialize pieces only when needed. The three stages shard progressively more, and the arithmetic at DP = 8 tells the story:

PER-GPU TRAINING STATE, BYTES PER PARAMETER (DP group of 8)

plain DP		16 B — everyone stores
ZeRO-1		5.5 B — optimizer m, v, master sharded $\div 8$
ZeRO-2		3.75 B — gradients sharded too (reduce-scatter)
ZeRO-3 / FSDP		2 B — weights as well: nothing whole anywhere

Sharding divides by the DP degree: at DP = 64, stage 3 holds 0.25 B/param — a 70B's states drop from 1.1 TB per GPU to ~18 GB.

The price is communication choreography: whole layers are re-assembled on demand, used, and thrown away — ever

Figure 14.2 — ZeRO's ladder at DP = 8. Stage 1 shards the optimizer's 12 bytes; stage 2 adds the gradient's 2; stage 3 — FSDP's home stage — shards the weights themselves, so no GPU ever holds a whole layer at rest. Activations are extra and unsharded (Chapter 15's job).

Stage 3 is the interesting one mechanically, because weights that are sharded at rest must exist wholly *somewhere* to be multiplied. FSDP's answer is a per-layer heartbeat:³ just before layer k 's forward, **all-gather** its shards into a briefly-whole layer; run the matmul; free it immediately; repeat at backward, then **reduce-scatter** the layer's gradients so each rank keeps only its shard. Peak memory holds one or two whole layers instead of a whole model, and — same trick as DP — the next layer's all-gather prefetches behind the current layer's compute. Total traffic is modest: roughly 1.5× plain DP's volume, in exchange for an 8×-or-more memory cut. The code stays two lines from Chapter 4:

```
# FSDP: ZeRO-3 with overlap, as a wrapper
from torch.distributed.fsdp import fully_shard, MixedPrecisionPolicy

for block in model.layers:
    fully_shard(block, mp_policy=MixedPrecisionPolicy(torch.bfloat16))
fully_shard(model)

# HSDP — the two-level refinement most real clusters want:
# shard WITHIN each node (all-gathers ride NVLink at 900 GB/s),
# replicate ACROSS nodes (only DP's gradient sync crosses the slow fabric)
from torch.distributed.device_mesh import init_device_mesh
mesh = init_device_mesh("cuda", (n_nodes, 8), mesh_dim_names=("replicate",
"shard"))
for block in model.layers: fully_shard(block, mesh=mesh)
# everything downstream — loop, checkpoints (sharded state dicts!), logging —
# is Chapter 4 with rank discipline. Full recipe: the companion repo.
```

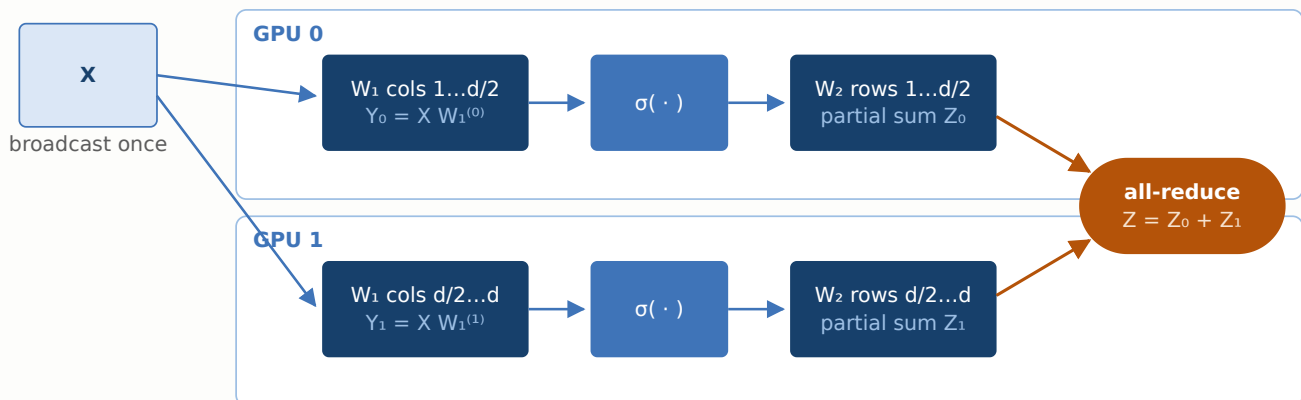
The wrapper granularity is the one real design decision — shard per transformer block, so gather units are layer-sized — and **HSDP** in the snippet is the §14.1 principle applied: the chatty all-gathers stay on NVLink, only the calm gradient sync crosses nodes. For dense models up to ~70B on a modern fabric, FSDP/HSDP *alone* is a complete answer, and it is where this book's audience should start: one mental model (DP with a memory trick), no model

surgery, and the same checkpointing story. The axes that follow exist for when it stops being enough — activations too large at long context, fabric too slow for stage-3 gathers, or a model too big for even sharded states plus a working set.

14.4 Tensor parallelism: cutting the matmul itself

Tensor parallelism goes *inside* the layer. Take the MLP's first matmul $Y = XW_1$ and split W_1 by **columns** across t GPUs: each computes $Y_i = XW_1^{(i)}$ — a valid slice of the output, no communication at all, because each output column depends on every input but only its own weight columns. Feed the pieces through the elementwise nonlinearity (still independent), then split the second matmul W_2 by **rows**: each GPU's Y_i meets exactly the rows of W_2 it needs, producing a *partial sum* of the final output — and one **all-reduce** adds the partials. That is Megatron's cell:⁴ column-split, nonlinearity, row-split, one all-reduce — and attention parallelizes even more naturally, since heads are already independent columns (each GPU takes h/t heads; the output projection is the row-split). Two all-reduces per layer per forward (MLP + attention), two more in backward.

THE MEGATRON CELL — one MLP across two GPUs



No communication until the cell's end — the column/row pairing makes the cuts cancel. Attention: identical, heads as natural columns. Cost: 2 all-reduces of a full activation per layer per direction — 43 GB/seq at 70B.

Figure 14.3 — The Megatron cell. Split the first matrix by columns (free), the second by rows (one all-reduce of partial sums). The activation traffic — twice per layer, every micro-batch, both directions — is why TP's home is the NVLink island: at 70B geometry it moves ~43 GB per 8K sequence per forward, which inter-node fabric cannot hide.

The mechanics deserve to exist in your hands once, because twenty-five lines dissolve the mystique:

```

# tensor parallelism by hand: the two halves of the Megatron cell
class ColumnParallelLinear(nn.Module):
    # W split by columns; no comm fwd
    def __init__(self, d_in, d_out, group):
        super().__init__()
        t = dist.get_world_size(group)
        self.w = nn.Parameter(torch.empty(d_in, d_out // t)) # my column slice
        self.group = group
    def forward(self, x):
        # every rank has full x
        # my slice of the output
        return x @ self.w
  
```

```

class RowParallelLinear(nn.Module):
    def __init__(self, d_in, d_out, group):
        super().__init__()
        t = dist.get_world_size(group)
        self.w = nn.Parameter(torch.empty(d_in // t, d_out))
        self.group = group
    def forward(self, x_shard):
        z = x_shard @ self.w
        dist.all_reduce(z, group=self.group)
        return z

mlp = nn.Sequential(ColumnParallelLinear(d, 4*d, tp), nn.SiLU(),
                    RowParallelLinear(4*d, d, tp))

```

Three practical riders. **TP ≤ 8 , and never across nodes** — the signature is too chatty for anything but NVLink, which is why TP degree equals the node's GPU count or less, everywhere, always. **Sequence parallelism** (the Megatron refinement, distinct from Chapter 21's context parallelism) notices that the norms and dropouts *between* cells don't need the full activation either — split them along the sequence and the all-reduces become reduce-scatter/all-gather pairs at identical cost but with activation memory divided by t : take it for free when your framework offers it.⁵ And TP divides the *working set* — weights, activations, KV — by t , which is what makes it the tool for models whose single layers outgrow a GPU, the regime FSDP alone handles poorly.

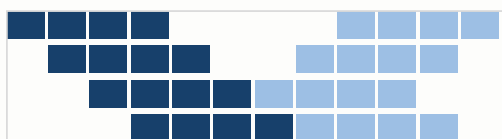
14.5 Pipeline parallelism: cutting the stack, fighting the bubble

The remaining cut is by **depth**: stage 1 holds layers 1–20, stage 2 holds 21–40, and so on; a micro-batch flows through the stages like a part down an assembly line, and all that crosses each boundary is one activation tensor, point-to-point — the *quietest* signature of any axis, which is why PP is the classic way to span nodes. The problem is not communication but **idleness**. Run one batch through four stages naively and each stage works a quarter of the time. GPipe's fix — split the batch into m micro-batches and stream them — shrinks the idle "bubble" to $\frac{p-1}{m+p-1}$ of the step:⁶ at 16 stages with 64 micro-batches, 19% of your cluster-seconds evaporate into geometry; push to 256 micro-batches and it's 5.5%, but each micro-batch shrinks toward kernel-inefficiency (Chapter 8). **1F1B** scheduling reorders the same work — each stage alternates one-forward, one-backward as soon as possible — leaving the bubble unchanged but capping in-flight activations at p micro-batches instead of m : the memory fix that made deep pipelines feasible.⁷ **Interleaving** gives each GPU several non-contiguous chunks of layers (virtual stages), cutting the bubble by the chunk count at the price of proportionally more boundary traffic.⁵

And then DeepSeek, whose V3 cluster had *no* NVLink-rich TP option worth paying for, decided the bubble itself was the enemy and built **DualPipe**:⁸ feed micro-batches into the pipeline *from both ends simultaneously*, pair each stage's forward of one stream with a backward of the other, and — the deeper trick — split each pairing so that computation of one chunk hides the *communication* of the other (in V3's case, the expert-parallel all-to-all of §14.6). The published schedule drives the bubble to near-zero at PP16 with modest micro-batch counts, at the cost of holding two directions' activations and a genuinely intricate schedule — state of the art, and the clearest recent proof that scheduling is still a research frontier, not a solved library call.

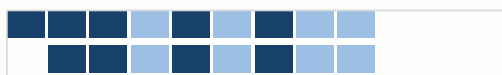
THREE SCHEDULES, FOUR STAGES, TIME → (navy = forward, blue = backward, empty = the bubble)

GPipe



bubble = $(p-1)/(m+p-1)$:
19% at $p=16$, $m=64$ — and all
 m micro-batches' activations held

1F1B



same bubble, but backwards start early:
only p micro-batches in flight — the memory fix

DualPipe



micro-batches enter from BOTH ends;
each cell pairs a forward with a backward, and
computation of one hides communication of the other

Strips are schematic; the structure — trailing bubble, early backwards, bidirectional pairing — is faithful.
DeepSeek-V3 ran PP16 with DualPipe, hiding its expert-parallel all-to-alls inside the paired cells.

Figure 14.4 — The bubble wars. GPipe streams micro-batches and pays a $(p-1)/(m+p-1)$ bubble; 1F1B pays the same bubble but bounds memory; DualPipe feeds both ends and pairs forward with backward so compute hides communication — near-zero bubble at V3's PP16, at the cost of the field's most intricate schedule.⁸

PP's two operational costs round out the picture. **Stage balance:** the bubble formula assumes equal stages, and the embedding + LM head make the first and last stages heavy — real configs shave layers off those stages, and imbalance shows up directly as bubble. And **global batch pressure:** the bubble math wants many micro-batches, which multiplies tokens per step — one more force (with DP) pushing frontier batch sizes up, to be reconciled with Chapter 17.

14.6 Expert parallelism: MoE's native axis

Mixture-of-experts models (Chapter 12) get an axis of their own, and it changes the default recipe. An MoE layer's experts are structurally independent — 256 separate MLPs of which each token uses 8 — so the natural cut is **one group of experts per GPU**, with a router upstream deciding where each token goes. The communication signature is new: an **all-to-all** — every GPU sends each of its tokens to the GPU holding its chosen experts, and a second all-to-all brings the outputs home — twice per MoE layer, in both passes.⁹ The volume is manageable (tokens, not weight matrices) but the pattern is dense and latency-sensitive, which makes EP the second-chattiest axis after TP — and yet, unlike TP, its payload *doesn't grow with the split*, which is why V3 could run EP64 across nodes and hide the traffic inside DualPipe's paired cells. For MoE models, EP largely *replaces* TP: the experts are already narrow, so cutting them further with TP buys little — the V3 recipe (EP64 + PP16 + ZeRO-1 DP, TP = 1) is the canonical demonstration.⁸ The tax EP does levy is Chapter 12's load-balancing problem wearing a systems costume: a hot expert is now a hot GPU, and the stragglers stall everyone.

14.7 Composing the axes: match the signature to the wire

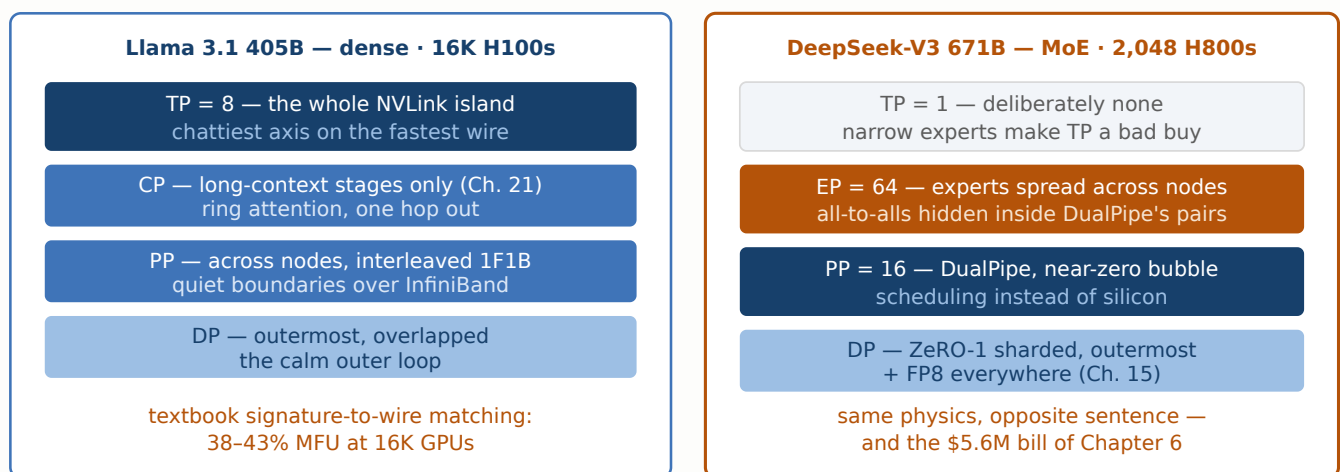
Now the payoff. Real recipes stack axes into a grid — a *device mesh* — and §14.1's principle decides the stacking order all by itself. **TP innermost**, on the NVLink island, because its per-layer activation all-reduces tolerate nothing slower.

CP next (Chapter 21), when sequences demand it — ring attention's neighbor-passing also loves fast wires but tolerates one hop more. **PP across nodes**, because a stage boundary is one quiet point-to-point tensor. **DP outermost**, because its single overlappable gradient sync is the most distance-tolerant traffic in the system — and at scale it rides ZeRO-1 sharding almost for free. **EP** slots in where the experts are, replacing TP for MoE. One worked example makes it muscle memory — our 70B on 256 H100s (32 nodes × 8):

$$\underbrace{\text{TP} = 8}_{\text{inside each node}} \times \underbrace{\text{PP} = 4}_{\text{groups of 4 nodes}} \times \underbrace{\text{DP} = 8}_{\text{across the groups}} = 256, \quad \frac{70\text{B} \times 16 \text{ B}}{8 \times 4} = 35 \text{ GB/GPU of state}$$

— comfortable under 80 GB with activations checkpointed, bubbles tamed by micro-batching, and every signature on the right wire. The two great frontier case studies bracket the design space:

TWO ANSWERS TO THE SAME PHYSICS



Neither recipe is "right": Llama's follows from dense-model + NVLink-rich fleet; V3's from MoE + a fabric worth avoiding TP on. Read recipe by asking what each choice says about the model's shape and the cluster's wires — that inference skill is this chapter's real p

Figure 14.5 — Two frontier compositions. Llama 3's 4D stack is the signature-matching principle executed by the book¹⁰; DeepSeek-V3's TP-free recipe is the same principle reaching a different conclusion because MoE changed the signatures.⁸ Between them they span most decisions you will ever make.

In modern PyTorch the mesh is an object you declare once, and every axis addresses its slice of it — worth seeing because it makes "4D" concrete rather than mystical:

```
# the mesh as code: 256 GPUs = DP 8 x PP 4 x TP 8 (outermost -> innermost)
from torch.distributed.device_mesh import init_device_mesh
mesh = init_device_mesh("cuda", (8, 4, 8), mesh_dim_names=("dp", "pp", "tp"))

tp_group = mesh["tp"].get_group() # 8 ranks sharing an NVLink island
pp_group = mesh["pp"].get_group() # 4 stage-neighbors across nodes
dp_group = mesh["dp"].get_group() # 8 replicas, outermost

# each subsystem takes its dimension:
# ColumnParallelLinear(..., group=tp_group)           (§14.4)
# PipelineSchedule(stages, group=pp_group)             (§14.5)
# fully_shard(block, mesh=mesh["dp"])                 (§14.3, ZeRO-1-style)
```

```
# rank (i, j, k) reads as: replica i, stage j, tensor-slice k – and the
# placement in (8, 4, 8) order is what puts TP on the fastest wires.
```

14.8 Debugging distributed training: the weekend-saving section

Distributed failures have a grammar, and knowing it converts panic into procedure. **The hang** is the signature failure: all ranks blocked in a collective, zero progress, then an NCCL timeout minutes later. Its cause is almost always one rank missing the party — it OOMed, hit a data error, or took a divergent code path (a conditional on rank, a dataset shard exhausted early) — so the procedure is: dump stacks on *every* rank (`py-spy` , or

`TORCH_NCCL_DUMP_ON_TIMEOUT`), find the rank that isn't in the collective, and read *its* error; the timeout message on the other 255 ranks is noise. **The desync** — loss diverging across replicas or NaNs on one rank — points at non-identical inputs (sampler seeding), non-deterministic kernels, or hardware (one bad HBM chip in a fleet is a matter of time; Chapter 4's 419 interruptions were mostly this). **The silent slowdown** — step time creeping up on one host — is a straggler: a thermally-throttled GPU, a slow NIC, a noisy neighbor; per-rank step-time histograms on the dashboard find it in minutes, and without them it hides for weeks. Rules that prevent whole bug classes: no rank-conditional control flow that touches collectives; log from rank 0 but *health-check from every rank*; make preemption and resume a drill (§4.3) before scale makes it an incident; and keep a 30-second single-node repro config, because most "distributed bugs" reproduce at world size 2.

The toolkit that operationalizes the grammar fits in one block — set it up before the first multi-node run, not after the first hang:

```
# distributed debugging: the standing kit
# --- environment (launcher-level) ---
# TORCH_NCCL_DUMP_ON_TIMEOUT=1          stacks + collective traces on hang
# TORCH_NCCL_TRACE_BUFFER_SIZE=2000     flight-recorder of recent collectives
# NCCL_DEBUG=WARN                       (INFO floods; WARN catches real trouble)
# TORCH_DISTRIBUTED_DEBUG=DETAIL        shape/dtype mismatch checks on collectives

# --- every-rank health beacon (the straggler-finder) ---
if step % 100 == 0:
    t = torch.tensor([step_time_ms], device="cuda")
    times = [torch.empty_like(t) for _ in range(world)]
    dist.all_gather(times, t)
    if rank == 0:
        worst = max(x.item() for x in times)
        if worst > 1.3 * median(x.item() for x in times):
            alert(f"straggler: rank {argmax} at {worst:.0f} ms") # find it in
                                                                # minutes, not
                                                                # weeks
# --- and the two drills ---
# 1) world-size-2 repro config for every "distributed" bug report
# 2) kill -9 one rank mid-run, resume from checkpoint, verify 100 losses match
```

14.9 What the big labs do

Table 14.1 — Shipped parallelism recipes.

RUN	RECIPE	THE LESSON
Megatron-LM (2019) ⁴	TP8 within DGX nodes	Defined the cell §14.4 teaches; TP ≤ node size ever since
MT-NLG 530B (2021) ⁵	TP8 × PP35 × DP, interleaved 1F1B	The 3D template the field standardized on
Llama 3.1 405B ¹⁰	TP8 × CP × PP × DP on 16K H100s, FSDP-style sharded DP	4D as routine engineering; 38–43% MFU at fleet scale
DeepSeek-V3 ⁸	EP64 × PP16 (DualPipe) × ZeRO-1 DP, TP=1, FP8	MoE rewrites the signatures; scheduling replaces silicon
The 7–70B mainstream	FSDP/HSDP, sometimes +TP at 34B+	Below frontier scale, sharded DP is usually the whole answer

14.10 Practical considerations

The mesh is a batch-size machine — notice what you set. Global batch = micro-batch × grad-accum × DP degree, and PP wants many micro-batches while DP multiplies everything: recipes routinely land at millions of tokens per step by accident. Fix the token budget per step *first* (Chapter 17), then solve the mesh under it — never the reverse.

Checkpoints are sharded artifacts now. Every axis changes what a checkpoint is: FSDP saves shards, TP saves slices, PP saves stages. Use your framework's distributed-checkpoint format, and — the rule that saves real tears — *decide and test your resharding story* (resume on a different GPU count) before the long run, not during the incident that makes it urgent. §4.3's resume-verification drill applies per-topology.

Know your expected MFU per config, and alarm on it. FSDP on one node: 45–55% of the single-GPU ideal is healthy. +TP: expect a few points down. +PP: subtract the bubble you computed. At Llama-scale 4D: 38–43% is the published frontier norm.¹⁰ A config that should do 42% doing 31% is a straggler, a fabric problem, or a bad overlap — Chapter 8's profiling finds which; the point here is to know the "should."

Common pitfalls

TP across nodes.

The classic first-cluster mistake: 43 GB per sequence per direction meets a 50 GB/s wire. TP stops at the NVLink boundary, no exceptions worth naming.

FSDP with tiny wrap units.

Sharding per-Linear instead of per-block makes thousands of small all-gathers that can't overlap. Wrap at the transformer block.

Unbalanced pipeline stages.

Embedding and LM head are layers too; a "even 20 layers per stage" split idles three stages on the fourth's schedule. Balance by measured time, not layer count.

Bubble math ignored.

PP8 with 8 micro-batches is a 47% bubble — half the cluster off. If you can't raise micro-batches (memory, batch budget), lower stages.

Collectives inside rank-conditional code.

One `if rank == 0: extra_eval()` that touches the model, and the fleet hangs at the next collective. Keep control flow identical across ranks.

Testing at world size 8, launching at 512.

Bubble ratios, batch semantics, and failure rates all change with scale. Rehearse at least one intermediate size, with a forced restart.

14.11 Decision framework

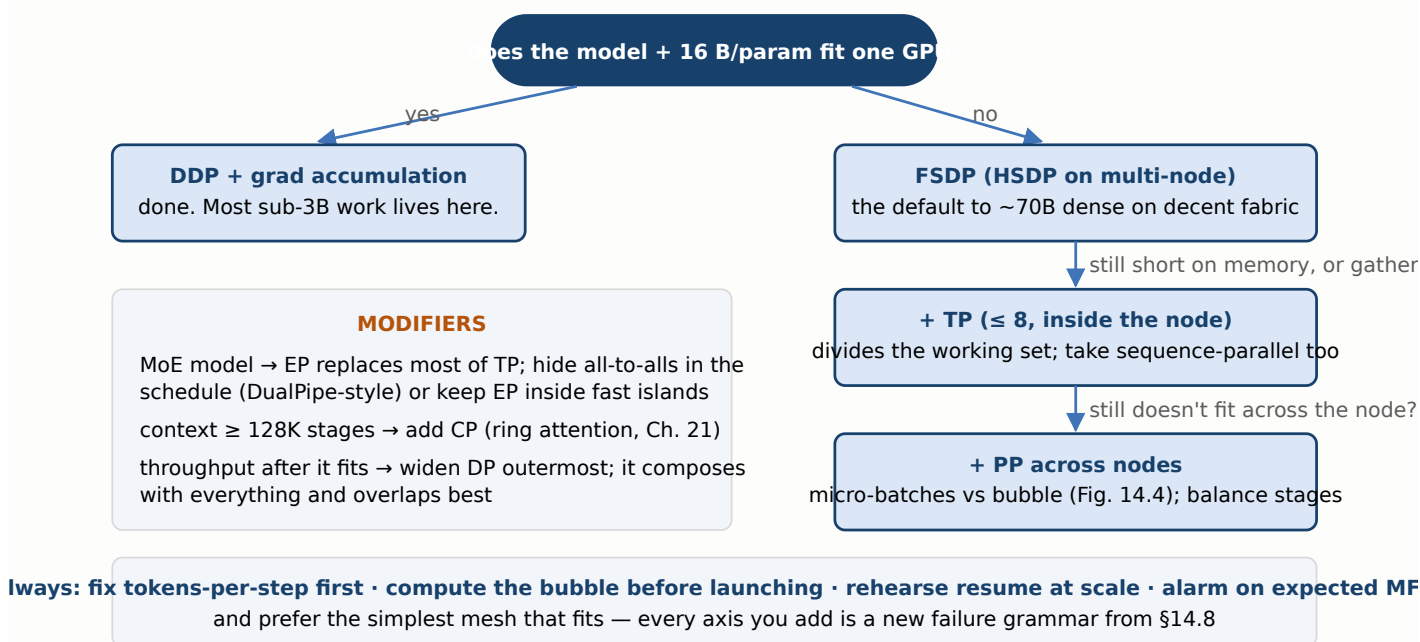


Figure 14.6 — Choosing a mesh. Escalate only when forced: DDP → FSDP/HSDP → +TP → +PP, with EP and CP as workload modifiers and DP always the outermost throughput dial. The bottom rail is the operational discipline that makes any of it survivable.

Cost-benefit, bluntly. DDP: an afternoon, and free. FSDP: a day including the checkpoint-format migration, and it unlocks everything to 70B — the single highest-value systems skill this book teaches. TP: real integration work (or Megatron/framework adoption) plus topology awareness — worth it exactly when single-layer working sets or FSDP gather times bind. PP: cheap to configure, expensive to *operate well* (balance, bubbles, restarts) — the axis to add last and remove first. DualPipe-class scheduling: frontier engineering with frontier payoff; admire it, adopt its lesson (overlap communication with paired compute), and buy it from a framework rather than writing it. **How this scales:** below 8 GPUs almost nothing here matters; at 8–64, FSDP/HSDP decisions dominate; at 256+, PP and stage balance appear; at frontier fleets, the recipe co-designs with the *building* — which is why Chapter 29's case study treats topology as an input, not a given. And one forward pointer: when the workload becomes *generate-then-train* — reinforcement learning's shape — the parallelism problem changes kind, not degree: two engines with opposite signatures sharing one set of weights. That story, and the verL machinery that solves it, is §25.3.

14.12 Summary

Distribution answers two different emergencies — the model doesn't fit, and the calendar doesn't fit — with four-and-a-half composable cuts, each defined by its communication signature. Data parallelism replicates and averages: ring all-reduce makes its traffic scale-free, overlap hides it, and it buys throughput but no memory. ZeRO/FSDP notices the replicas' redundancy and shards the 16 bytes per parameter down the stages — optimizer, gradients, weights — reassembling layers on demand; with HSDP keeping gathers on NVLink, it is the complete answer for most dense models to ~70B. Tensor parallelism cuts every matmul by column and row so one all-reduce per cell reunites the pieces

— the chattiest signature, confined to the node, and the tool that divides working sets. Pipeline parallelism cuts by depth for the quietest traffic and pays in bubbles: $(p-1)/(m+p-1)$ for GPipe, memory-bounded by 1F1B, near-erased by DualPipe's bidirectional pairing of forwards with backwards that hides communication inside compute. Expert parallelism gives MoE its native axis — all-to-alls in place of weight cuts — and largely retires TP there. Composition is signature-to-wire matching, bracketed by Llama 3's textbook 4D and DeepSeek-V3's TP-free counter-recipe; debugging is a grammar (hangs mean one missing rank; read *its* error); and the discipline is constant: fix the token budget first, compute the bubble before launch, rehearse resume, alarm on the MFU you should be getting, and add no axis you aren't forced to.

References

1. Sergeev, A., & Del Balso, M. (2018). Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. arXiv:1802.05799. Ring all-reduce for gradient averaging (building on Baidu's ring implementation).
2. Rajbhandari, S., Rasley, J., Ruwase, O., & He, Y. (2020). ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. *SC20*. arXiv:1910.02054. The three sharding stages of §14.3.
3. Zhao, Y., et al. (2023). PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *VLDB*. arXiv:2304.11277. The all-gather/reduce-scatter heartbeat, wrapping, and hybrid sharding.
4. Shoeybi, M., et al. (2019). Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053. The column/row cell of §14.4.
5. Narayanan, D., et al. (2021). Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. *SC21*. arXiv:2104.04473. Interleaved 1F1B, sequence parallelism, and the 3D recipe (MT-NLG).
6. Huang, Y., et al. (2019). GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism. *NeurIPS*. arXiv:1811.06965. Micro-batching and the bubble formula.
7. Narayanan, D., et al. (2019). PipeDream: Generalized Pipeline Parallelism for DNN Training. *SOSP*. arXiv:1806.03377. The 1F1B family's origin.
8. DeepSeek-AI (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437. DualPipe's bidirectional schedule with compute–communication pairing; the $EP64 \times PP16$, TP-free recipe on 2,048 H800s.
9. Lepikhin, D., et al. (2021). GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. *ICLR*. arXiv:2006.16668. Expert parallelism's all-to-all dispatch/combine.
10. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. The 4D composition (TP, CP, PP, DP) on 16K H100s and its 38–43% BF16 MFU.

Related: *Chapter 15 — Fitting more into memory* (the activation side these meshes assume) · *Chapter 21 — Extending context* (CP joins the mesh) · *Chapter 25 — RL for reasoning* (§25.3: the generate-then-train parallelism problem and veRL) · *Chapter 29 — the case study*. · [Training Efficient LLMs](#)